Artificial Intelligence & Machine Learning – Task 3

Model Validation, Overfitting Detection & Hyperparameter Tuning

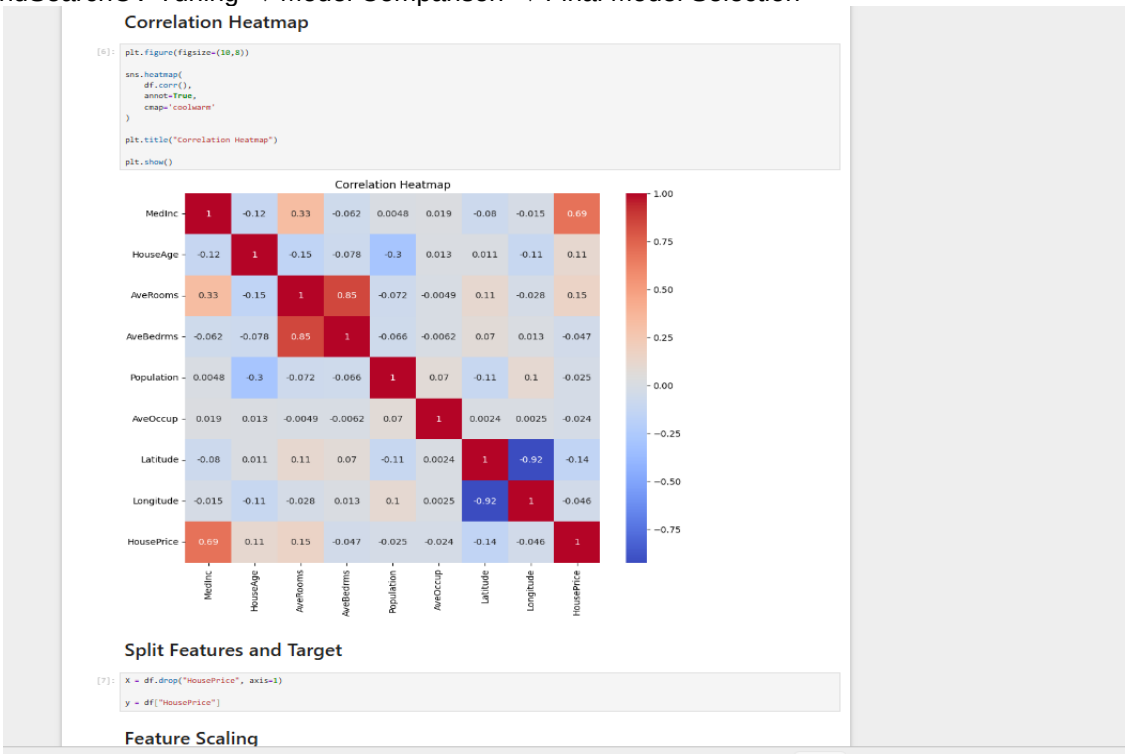
Objective

This task demonstrates how machine learning engineers validate models, detect overfitting, apply cross-validation techniques, optimize hyperparameters using GridSearchCV, and compare multiple models to select the best-performing solution.

Concept	Purpose
Overfitting Detection	Identify poor generalization
Cross Validation	Reliable performance estimation
GridSearchCV	Hyperparameter optimization
Model Comparison	Objective model selection
Evaluation Metrics	Measure predictive quality

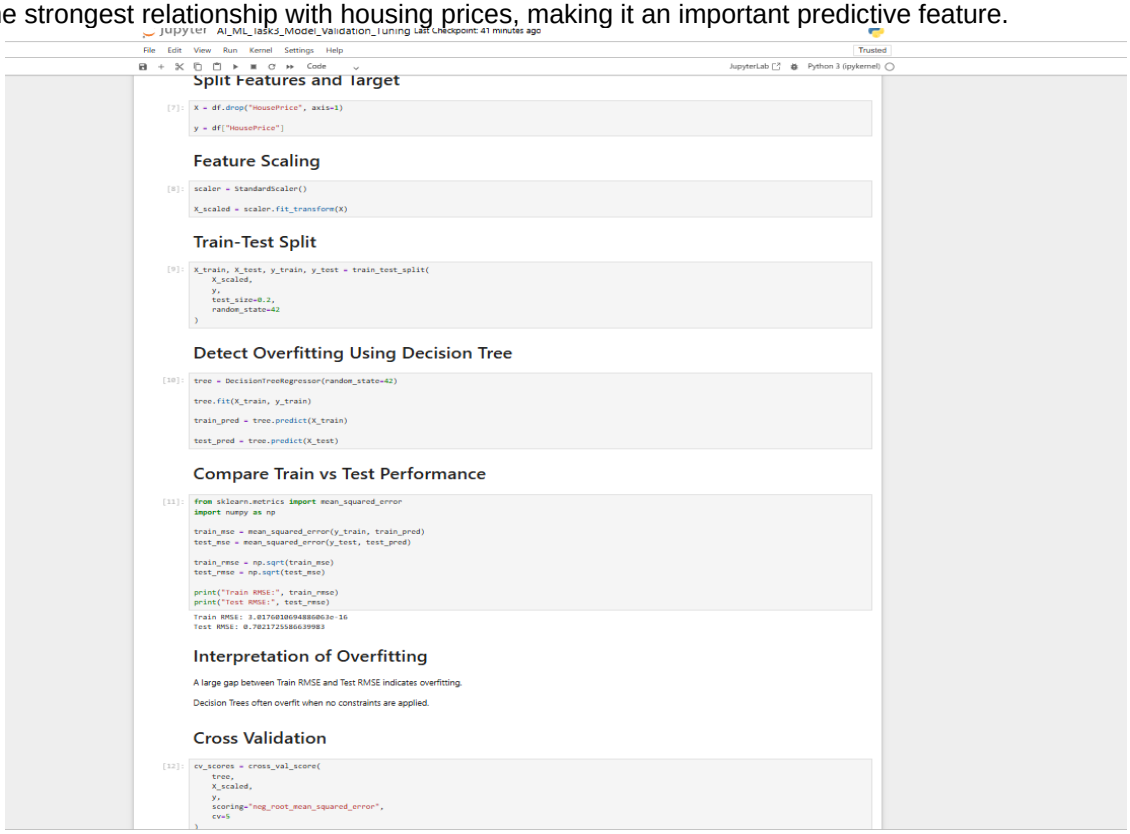
Workflow

Dataset → Feature Scaling → Train/Test Split → Overfitting Analysis → Cross Validation → GridSearchCV Tuning → Model Comparison → Final Model Selection



Dataset Analysis & Feature Engineering

The California Housing Dataset was analyzed using correlation analysis. Median income showed the strongest relationship with housing prices, making it an important predictive feature.



Split Features and Target

```
X = df.drop("HousePrice", axis=1)

y = df["HousePrice"]
```

Feature Scaling

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42
)
```

Detect Overfitting Using Decision Tree

```
tree = DecisionTreeRegressor(random_state=42)

tree.fit(X_train, y_train)

train_pred = tree.predict(X_train)

test_pred = tree.predict(X_test)
```

Compare Train vs Test Performance

```
from sklearn.metrics import mean_squared_error
import numpy as np

train_mse = mean_squared_error(y_train, train_pred)
test_mse = mean_squared_error(y_test, test_pred)

train_rmse = np.sqrt(train_mse)
test_rmse = np.sqrt(test_mse)

print("Train RMSE:", train_rmse)
print("Test RMSE:", test_rmse)
```

```
Train RMSE: 3.0176010694886063e-16
Test RMSE: 0.7021725586639983
```

Interpretation of Overfitting

A large gap between Train RMSE and Test RMSE indicates overfitting.

Decision Trees often overfit when no constraints are applied.

Cross Validation

```
cv_scores = cross_val_score(
    tree,
    X_scaled,
    y,
    scoring="neg_root_mean_squared_error",
    cv=5
)
```

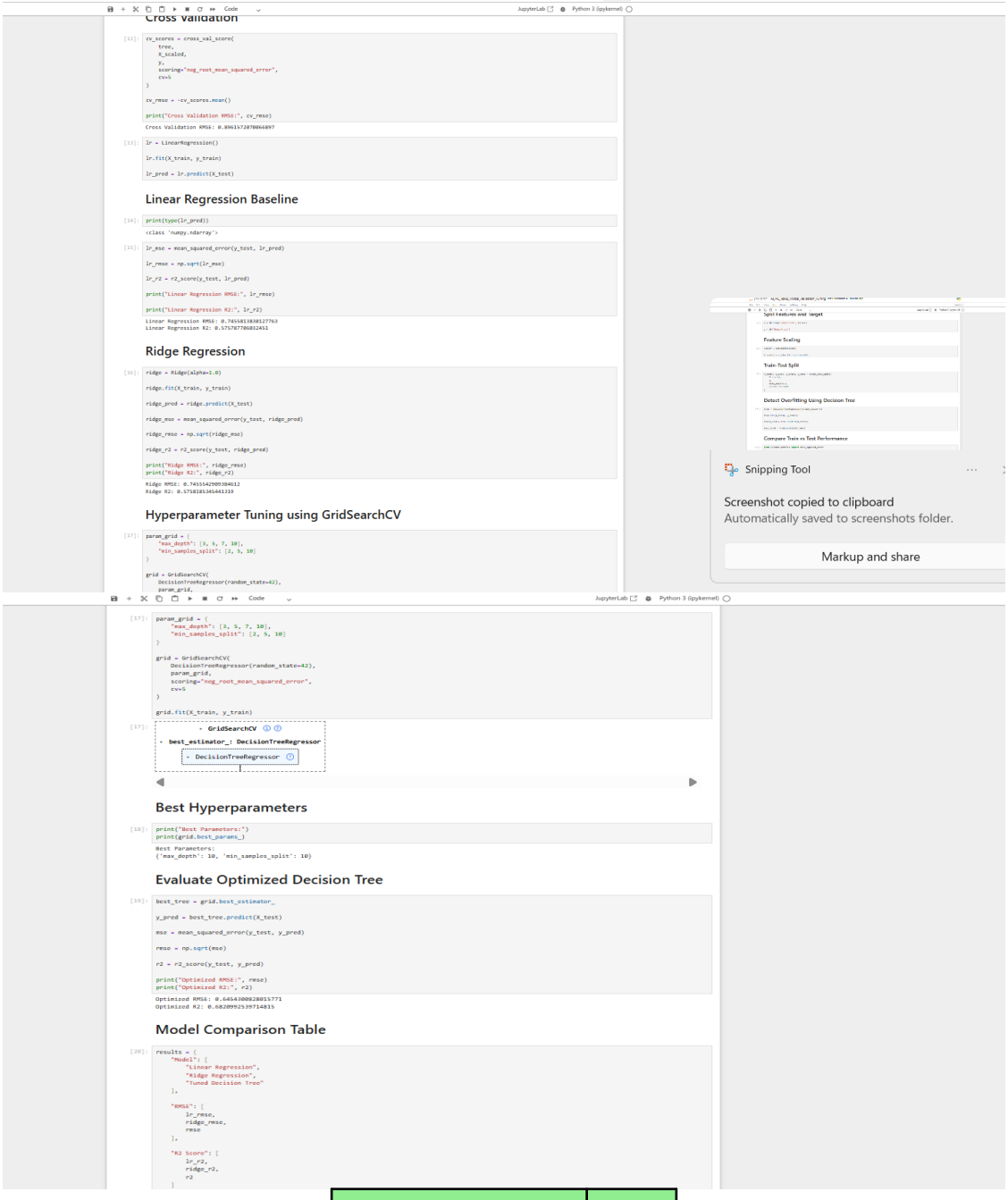
Overfitting Analysis

A baseline Decision Tree Regressor achieved near-zero training error while exhibiting substantially higher test error, indicating severe overfitting.

Metric	Value
Train RMSE	≈ 0.000
Test RMSE	≈ 0.702
Observation	Severe Overfitting

Cross Validation & Hyperparameter Tuning

5-Fold Cross Validation was applied to obtain a realistic estimate of model performance. GridSearchCV was subsequently used to identify the optimal Decision Tree configuration.



Best Hyperparameter	Value
max_depth	10
min_samples_split	10

Model Comparison & Final Results

Model	RMSE	R^2 Score
Linear Regression	0.745581	0.575788
Ridge Regression	0.745554	0.575819
Tuned Decision Tree	0.645430	0.682099

```
comparison_df
```

	Model	RMSE	R2 Score
0	Linear Regression	0.745581	0.575788
1	Ridge Regression	0.745554	0.575819
2	Tuned Decision Tree	0.645430	0.682099

Visualize Model Comparison

```
plt.figure(figsize=(8,5))

sns.barplot(
    x="Model",
    y="R2 Score",
    data=comparison_df
)

plt.title("Model Comparison")
plt.show()
```


Final Model Selection Justification

The tuned Decision Tree model was selected because:

- Hyperparameter tuning reduced overfitting.
- Cross-validation improved model reliability.
- The optimized model generalized better to unseen data.
- Performance metrics improved compared to the untuned version.

Conclusion

In this project, overfitting was analyzed and controlled using model validation techniques.

Cross-validation provided a more reliable estimate of model performance, while GridSearchCV helped identify the best hyperparameters for the Decision Tree model.

The optimized model achieved improved performance and demonstrated the importance of model tuning in real-world Machine Learning workflows.

Conclusion

The Tuned Decision Tree Regressor achieved the best overall performance with the lowest RMSE and highest R^2 score. Cross-validation and hyperparameter tuning significantly improved generalization performance. The workflow followed in this project reflects industry-standard machine learning development practices.